

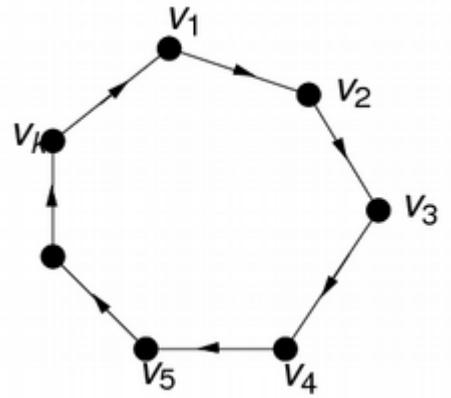
PROJETO E ANÁLISE DE ALGORITMOS – 01A – 2022.2

[Notas de aula: detecção de bipartição, ciclos e ordenação topológica](#)

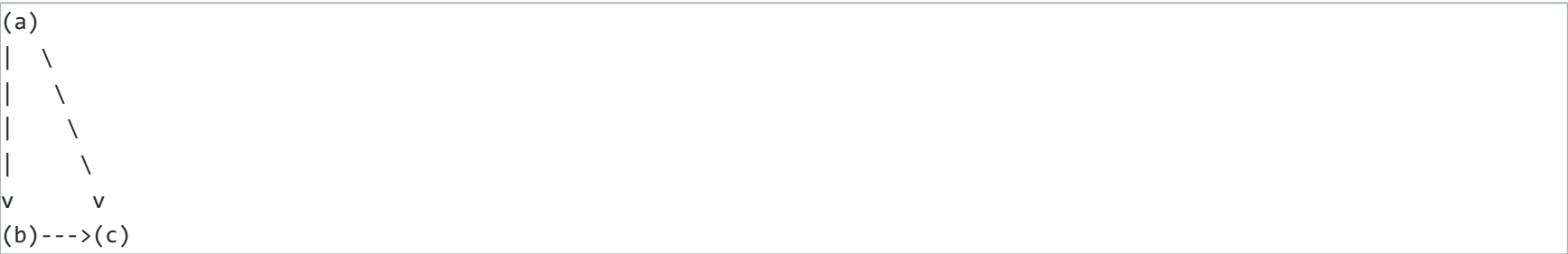
Nesta aula vamos continuar estudando aplicações da busca em profundidade (DFS) em grafos direcionados: detecção de bipartição, detecção de ciclo, e ordenação topológica.

Detecção de ciclos

Em grafos direcionados:



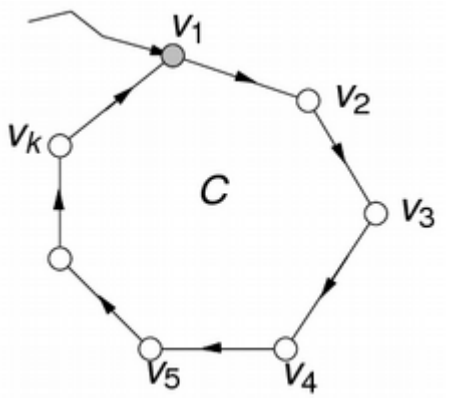
Obs.: a situação abaixo não forma ciclo direcionado.



Teorema: Um grafo direcionado tem ciclo sse uma DFS encontra aresta de retorno.

($\Leftarrow$) Se $((u,v))$ é uma aresta de retorno, então o caminho de (v) até (u) na ABP, juntamente com $((u,v))$, forma um ciclo.

($\Rightarrow$) Seja (C) um ciclo, e (v_1) o 1o vértice de (C) descoberto na DFS. Pelo teorema do caminho branco, (v_k) será descendente de (v_1) na ABP. Portanto, $((v_k,v_1))$ é aresta de retorno.



Em grafos não direcionados:

Podemos modelar um grafo não direcionado como um direcionado, para isso trocando cada aresta $(\{u,v\})$ por duas arestas $((u,v))$ e $((v,u))$. Portanto, podemos também encontrar ciclos em grafos não direcionados usando arestas de retorno.

Detecção de grafos bipartidos e ciclos de comprimento ímpar (grafos não direcionados)

Teorema: Um grafo é bipartido sse não contém ciclo de comprimento ímpar.

($\Rightarrow$) (contrapositiva) Se tem ciclo de comprimento ímpar, então o número cromático é pelo menos 3. Porém, todo grafo bipartido tem número cromático no máximo 2.

($\Leftarrow$) (contrapositiva) Considere uma ABL qq inciando em (s) . Coloque em (A) o vértice (s) e to distância par de (s) . Coloque em (B) todos com distância ímpar de (s) . Se não for bipartido, então existe are $((u,v))$ onde $(dist(u))$ e $(dist(v))$ são ambos pares ou ímpares (contrapositiva: se não existe aresta entre dois com dist de mesma paridade, então os conjuntos A e B formam uma bipartição - ou seja, o grafo é bipartido). Seja distância do ancestral comum de (u) e (v) até (s) . Temos que $((dist(u)-c)+(dist(v)-c) =) (dist(u)+dist(v)-2c)$ par. Logo, a aresta $((u,v))$ forma um ciclo ímpar.



Projete algoritmo que retorna uma bipartição ou um ciclo de comprimento ímpar. Força-bruta para bipartição: para subconjunto (S) de (V) , teste se toda aresta vai de (S) até $(V-S)$. Complexidade $(O(2^V \cdot E))$.

Outra abordagem é adaptar a busca em largura da forma como foi empregada para provar a volta do teorema ac Para cada aresta explorada na BL, verifique se liga vértices onde os d's possuem a mesma paridade. Se sim, retorne formado por esta aresta. Se for até o final da busca sem encontrar, retorne os vértices com d par em uma partição, vértices com d ímpar na outra partição.

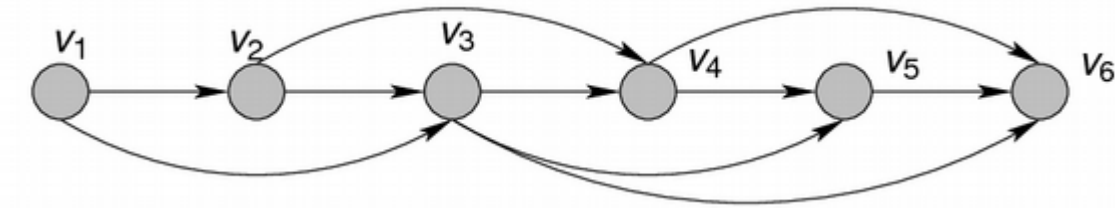
```
DETECÇÃO_BIPARTIDO(Adj): // Adaptação da BFS
// INICIALIZAÇÃO
escolha um vértice qualquer s          (*) O(1)
para cada vértice u de G (excluindo o s)  O(V)
    cor[u] <- branco; d[u] <- oo; pi[u] <- NULO  O(V)
cor[s] <- cinza; d[s] <- 0; pi[s] <- NULO      O(1)

declare uma fila vazia Q                O(1)
insira s na fila Q                      O(1)
enquanto Q não é vazia                  O(V)
    remova o próximo elem. da fila Q e o armazene em u  O(V)
    para cada vértice v em Adj[u]          O(V+E)
        se cor[v] = branco                O(V+E)
            cor[v] <- cinza; d[v] <- d[u] + 1; pi[v] <- u  O(V+E)
            insira v na fila Q              O(E)
        senão
            se paridade(d[u]) = paridade(d[v]), (*) O(E)
                retorne IMPRIME-CICLO-IMPAR(u,v) (*) O(E)
retorne (A,B), onde A são os nós com d par, e B ímpar. (*) O(1)

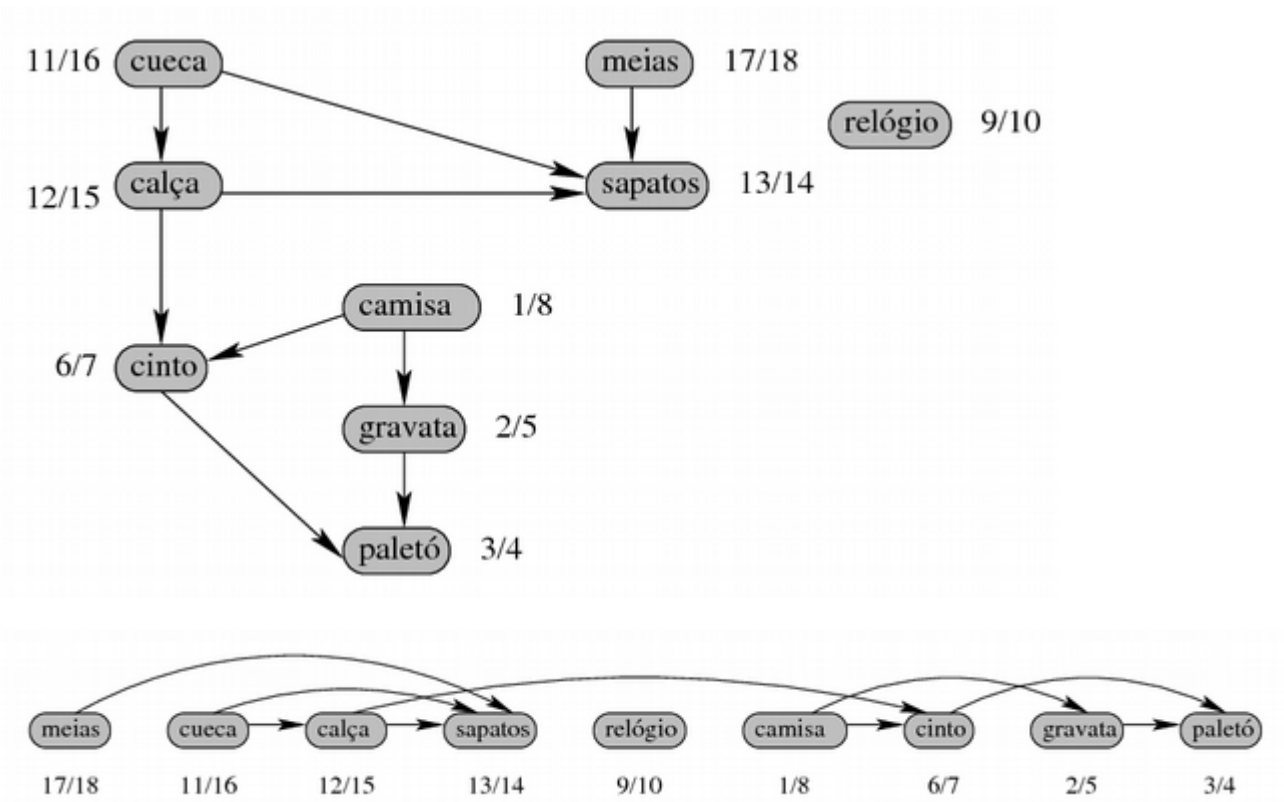
IMPRIME-CICLO-IMPAR(u,v) // O(V)
    imprime v
    w <- u
    enquanto w != v
        imprime w
        w <- pi[w]
```

Ordenação topológica

Uma ordenação topológica do grafo direcionado (G) é uma ordenação dos vértices de modo que toda aresta "v esquerda para a direita".

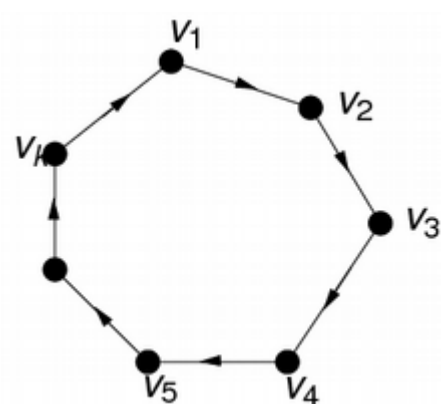


Exemplo:



Teorema: Um grafo direcionado admite ordenação topológica SSE não possui ciclo.

($\Rightarrow$) (contrapositiva) Em um ciclo, para qualquer ordenação dos vértices teremos aresta "voltando".



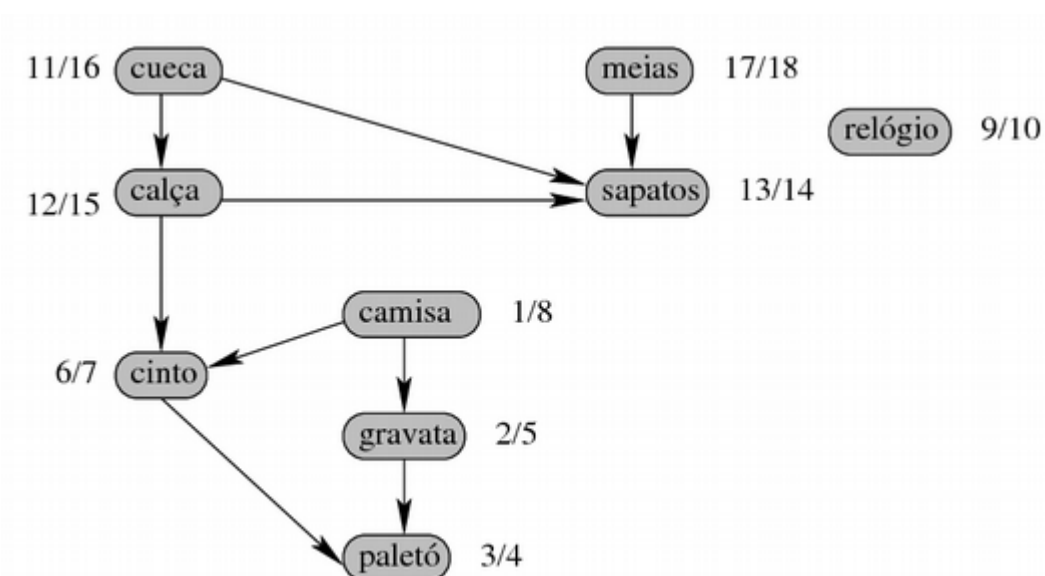
($\Leftarrow$) Para mostrar a volta construiremos um algoritmo eficiente para encontrar ordenação topológica e qualquer grafo sem ciclos (acíclico).

Definição:

Fonte: vértice com grau de entrada zero.

Sorvedouro: vértice com grau de saída zero.

Exemplo:



Lema: Todo grafo direcionado sem ciclos possui fonte e sorvedouro.

Por contradição, assuma que não tem fonte, de modo que todo vértice tem um predecessor. Como o número de vértices é finito, seguindo os predecessores vamos em algum momento encontrar novamente algum vértice, formando ciclo (contradição).

Argumento similar para existência de sorvedouro.

Lema: A remoção de vértices e arestas mantém um grafo acíclico sem ciclos.

Algoritmo: Uma fonte é um vértice seguro para iniciar a ordenação topológica, pois não produzirá arestas "voltando". A remoção de uma fonte, e arestas incidentes, mantém o grafo sem ciclos. Portanto, após selecionar uma fonte e removê-la do grafo, podemos calcular recursivamente a ordenação topológica do grafo que sobrou.

PRÉ-PROCESSAMENTO: determine o grau de entrada d de cada vértice. $O(V+E)$.
Seja S o conjunto dos vértices com grau 0 (obtido no cálculo dos graus).

ORDENAÇÃO-TOPOLÓGICA(Adj):
se G tem apenas o vértice u , imprima u e retorne $O(1)$
senão,
 remova um elemento u qq de S (fonte) $O(1)$
 imprima u $O(1)$
 para todo v em $Adj[u]$, $|Adj[u]|+1$
 remova v de $Adj[u]$ $|Adj[u]|$
 decremente $d[v]$ $|Adj[u]|$
 se $d[v] = 0$, insira v em S $|Adj[u]|$
 remova u de G $O(1)$
ORDENAÇÃO-TOPOLÓGICA(Adj) $T(V-1)$

A solução desta recorrência é $O(V+E)$.

De acordo com o teorema abaixo, outra opção é realizar uma DFS e em seguida imprimir os vértices em ordem dec de $\backslash(f[]\backslash)$.

Teorema: Para toda arestas $\backslash((u,v)\backslash)$ de um grafo acíclico, temos $\backslash(f[u] > f[v]\backslash)$.

- $\backslash((u,v)\backslash)$ não pode ser aresta de retorno, pois formaria ciclo.
- Se $\backslash((u,v)\backslash)$ é de árvore, $\backslash(v\backslash)$ é descendente de $\backslash(u\backslash)$, portanto $\backslash(f[u] > f[v]\backslash)$.
- Se $\backslash((u,v)\backslash)$ é de avanço ou cruzada, então $\backslash(v\backslash)$ era preto quando $\backslash((u,v)\backslash)$ foi avaliada, logo $\backslash(f[u] > f[v]\backslash)$.

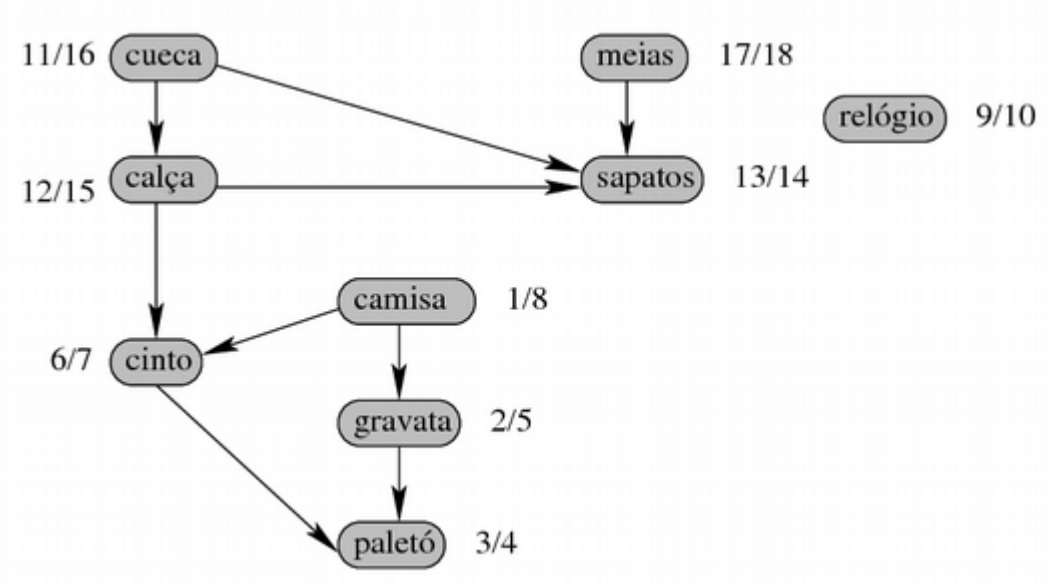
```
seja ord_topologica uma lista encadeada vazia  (*)
tempo <- 0

DFS-VISIT(Adj,u)
  cor[u] <- cinza
  d[u] <- tempo <- tempo + 1
  para cada v em Adj[u]
    se cor[v] = branco
      pi[v] <- u
      DFS-VISIT(Adj,v)
    se cor[v] = cinza, retorne FALSO  (*)
  cor[u] <- preto; f[u] <- tempo <- tempo + 1
  insira u no início de ord_topologica  (*)
  retorne VERDADEIRO  (*)

DFS(Adj)
  para cada vértice u
    cor[u] <- branco; pi[u] <- NULO
  para cada vértice u
    se cor[u] = branco
      se DFS-VISIT(Adj,u) = FALSO, retorne FALSO  (*)
  retorne VERDADEIRO  (*)
```

Exercício

- 1) Aplique o algoritmo p/ detecção de grafo bipartido ou ciclo ímpar no grafo abaixo (ignorando direção das aresta indique a bipartição (ou ciclo ímpar) formado.
- 2) Aplique uma DFS no grafo abaixo e imprima os vértices em ordem decrescente de $\backslash(f[]\backslash)$. Verifique a formação c ordenação topológica.



[◀ Notas de aula: busca em profundidade \(DFS\)](#)

Seguir para...

[Notas de aula: componentes fortemente conexos ▶](#)

©2020 – Universidade Federal do Ceará – Campus Quixadá.

Todos os direitos reservados.

Av. José de Freitas Queiroz, 5003

Cedro – Quixadá – Ceará CEP: 63902-580

Secretaria do Campus: (88) 3411-9422

 [Baixar o aplicativo móvel.](#)